

Finance Research Agent: A Multi-Step LLM Chain for Equity Analysis

1 Problem Statement

I built an agent to solve automated equity research: given a natural-language investment query such as “Should I invest in TCS for value investing?” it produces a structured, professional-grade research report with financial health scores, sentiment analysis, risk assessment, and a buy/hold/sell verdict. I realized that a naive single-prompt approach would fail for several reasons. First, the context window would be overwhelmed by the volume of raw financial metrics, news snippets, and historical price data needed for a thorough analysis. Second, a monolithic prompt gives the model no opportunity to reason incrementally; it must simultaneously parse intent, evaluate fundamentals, read news, gauge sentiment, and synthesise risk all at once, which degrades accuracy. Third, a single prompt cannot interleave live tool calls — fetching real-time stock data from Yahoo Finance and recent news from DuckDuckGo — which are essential because LLM training data is static and rapidly stale in financial markets. By decomposing the task into a directed chain of specialised steps, each step receives only the information it needs, can invoke external tools at precise moments, and passes structured intermediate results forward, I improved both coherence and factual grounding.

2 Chain Design

I designed a pipeline comprising seven steps arranged in a directed acyclic graph.

Step 1 — Query Parsing receives the raw user query and I have it produce a structured JSON object containing the ticker symbol, company name, analysis type, time horizon, and confidence level. This separation isolates intent understanding from downstream reasoning and ensures every subsequent step has an unambiguous symbol to query.

Step 2 — Stock Data Fetch is the first tool step. I implemented it to take the parsed ticker and call `yfinance` to retrieve live fundamentals (price, P/E, market cap, debt ratios, revenue growth, ROE) and a one-month price history. It returns a structured dictionary that acts as the single source of truth for all financial metrics in later steps.

Step 3 — Financial Analysis consumes the stock fundamentals and I designed it to produce a scored JSON assessment covering valuation, financial health, and growth, each on a 1–10 scale, plus qualitative summaries, strengths, concerns, and peer context. I kept this separate from sentiment to prevent the model from conflating hard accounting data with soft news narratives.

Step 4 — News Fetch is the second tool step. I built it to query DuckDuckGo for recent articles using the company name and ticker, returning a list of headlines, snippets, and URLs. I isolated news retrieval to ensure the subsequent sentiment step reasons only over actual, current coverage rather than stale training memories.

Step 5 — Sentiment Analysis ingests the news articles and I designed it to produce a

structured sentiment verdict (bullish to very bearish, score -5 to $+5$), key themes, catalysts, risks, and confidence. I made this step distinct from Step 3 because sentiment is orthogonal to fundamentals; a financially strong firm can suffer from negative news, and vice-versa.

Step 6 — Risk Assessment synthesises the outputs of Steps 3 and 5 alongside raw market data to produce a risk level, risk score, and investor suitability profile. I deliberately placed this step after both fundamental and sentiment analyses so that risk is evaluated from both quantitative and qualitative angles.

Step 7 — Report Generation consumes *all* prior outputs and writes the final markdown report in a fixed seven-section structure. I made this a dedicated final step, rather than appending a template to Step 6, to ensure consistent formatting and allow the model to focus purely on prose synthesis without being burdened by the analytical work already completed.

3 Tool Integration

I integrated two external tools. **Yahoo Finance via `yfinance`** provides real-time stock fundamentals and price history. I chose it because it requires no API key, supports both US and Indian NSE tickers (via the `.NS` suffix), and returns a rich set of accounting metrics in a single call. Its output enters the chain at Step 2 and is subsequently referenced in Steps 3, 5, 6, and 7. **DuckDuckGo search via `ddgs`** fetches recent news articles without authentication. I selected it because it is free, lightweight, and does not require registering for a search API key. Its output enters at Step 4 and feeds directly into Step 5. I wrapped both tools in helper functions that return standardised dictionaries with `success`, `data/articles`, and `error` keys, so the chain can degrade gracefully when a tool fails.

4 Limitations

The most severe limitation I encountered is rate limiting on the free-tier Gemini API. The daily quota for `gemini-2.5-flash` is twenty requests per project, and because my chain makes five LLM calls per query (Steps 1, 3, 5, 6, and 7), a single run consumes 25% of the daily budget, allowing only four full runs per day. Retry logic with exponential backoff mitigates transient 503 errors, but when the daily quota is exhausted no amount of waiting helps; the chain simply aborts. I added a fallback to `gemini-2.0-flash`, but that model shares the same project quota, so the fallback often fails too.

Another failure mode is non-existent ticker symbols. Originally, `yfinance` returned an empty dictionary for invalid tickers without raising an exception, which the chain treated as a success. The LLM then hallucinated an entire report based on null data, assigning arbitrary scores and fabricating a disclaimer date. I fixed this by adding a validation guard after the tool call and an early exit in the main runner, but the underlying issue — that `yfinance` silently returns empty data — remains a fragility.

Date hallucination was a recurring bug. The system prompt instructed the model to write “Date: [today’s date]”, but LLMs lack real-time awareness and defaulted to November 2023, their approximate training cutoff. I resolved this by injecting the current system date via Python’s `datetime` module before each report generation prompt.

Finally, console output became unreadable after I stripped non-ASCII characters from source files. Decorative box-drawing characters were replaced with empty strings, collapsing the visual layout.

5 Reflection

If I had more time, I would implement three major improvements. First, I would add an LRU cache for tool outputs so that repeated queries for the same ticker within a session do not trigger redundant API calls, drastically reducing the likelihood of exhausting the daily LLM quota. Second, I would replace the synchronous chain with an asynchronous runner so that Steps 3 and 4 (financial analysis and news fetch) could execute in parallel, cutting latency significantly. Third, I would build a lightweight web UI using a framework such as Streamlit or Gradio so that users can input queries, visualise intermediate step outputs, and download reports without touching the command line. Additionally, I would investigate more robust ticker validation — querying a known exchange API or maintaining a local ticker database — so that non-existent symbols are caught before any network request is made. Finally, I would add structured logging and metrics collection to diagnose exactly which steps fail most often and under what conditions, enabling data-driven iteration rather than reactive debugging.

References

- [1] Ranaroussi, A. *yfinance*: Yahoo! Finance market data downloader. <https://github.com/ranaroussi/yfinance>
- [2] de Freitas, N. *ddgs*: DuckDuckGo Search for Python. https://github.com/deedy5/duckduckgo_search
- [3] Google. *Gemini API*. <https://ai.google.dev/gemini-api/docs>